

COLLEGE CODE: 9618

COLLEGE NAME: Ponjesly college of engineering

DEPARTMENT: Computer science and engineering

STUDENT NMID: CB8573C069EFD16B6294C8216939B3BA

ROLL NO: 62

DATE: 13-10-2025

Technology: HTML, CSS, JAVA SCRIPT

Project name: Recipe Book App

Completed the project name as Hackathon,

Submitted by:(J. J. Jenifer (TL), N. Fathima Najiya, I. Asmitha, M. Madhu Midha, M. N. Arshi Fathima)

NAME: J. J. Jenifer

Mobile no: 6380787025

Recipe Book App

<https://github.com/Jenifer-lulu/IBM-FE-E-Commerce-Product-Page.git>

1. Project Overview & Objectives

Problem Statement

In today's digital lifestyle, people often discover and create recipes but lack a centralized, customizable platform to store, organize, and share them. Traditional cookbooks are static, and online recipes are scattered across blogs and videos, making it hard to personalize or retrieve them efficiently. The Recipe Book App aims to provide a real-time web application that allows users to manage their personal recipe collections, search by ingredients or categories, and optionally share with others.

Key Features

- Add, edit, and delete recipes with ingredients, steps, and media
- Search and filter recipes by category, ingredients, or tags
- Responsive and user-friendly interface
- Error handling for invalid inputs and missing fields
- Option to save favorite recipes or view recent ones
- Optional user authentication for personalized collections

Expected Outcome

A fully functional, responsive Recipe Book web application that allows users to manage their recipes efficiently. It will feature a clean UI, support multimedia content, and work seamlessly across devices.

2. Technology Stack & Environment Setup

- **Front-End Framework:** React.js for dynamic UI
- **Backend:** Node.js with Express.js for RESTful API handling
- **Database:** MongoDB (via MongoDB Atlas) for storing recipes and user data
- **External Tools:** Cloudinary or Firebase for image hosting (optional)
- **Environment:** Visual Studio Code, Postman, GitHub, Netlify/Vercel for deployment

3. API Design & Data Model

Planned REST Endpoints

- GET /api/recipes – Fetch all recipes
- GET /api/recipes/:id – Fetch a single recipe
- POST /api/recipes – Add a new recipe
- PUT /api/recipes/:id – Update a recipe
- DELETE /api/recipes/:id – Delete a recipe
- GET /api/favorites – Get user's favorite recipes (optional)
- POST /api/favorites – Save a recipe to favorites (optional)

Request/Response Format

Request Format:

```
{  
  "title": "Tomato Soup",  
  "ingredients": ["Tomatoes", "Garlic", "Salt"],  
  "steps": "Boil tomatoes, blend, season.",  
  "category": "Soup",  
  "imageUrl": "https://..."  
}
```

Response Format:

```
{  
  "id": "abc123",  
  "title": "Tomato Soup",  
  "ingredients": [...],  
  "steps": "...",  
  "category": "Soup",  
  "imageUrl": "...",  
  "createdAt": "2025-10-18T10:00:00Z"  
}
```

Database Schema:

```

{
  "userId": "string",
  "recipes": [
    {
      "title": "string",
      "ingredients": ["string"],
      "steps": "string",
      "category": "string",
      "imageUrl": "string",
      "createdAt": "Date"
    }
  ],
  "favorites": ["recipeId"]
}

```

4. Front-End UI/UX Plan

Wireframes

1. **Home Page** – List of recipes with search/filter options
2. **Recipe Detail Page** – Full recipe view with image and steps
3. **Add/Edit Recipe Page** – Form for creating or updating recipes
4. **Favorites Section** – User's saved recipes (optional)

Navigation Flow

Home → View Recipe → Add/Edit Recipe → Save → View Favorites → Return Home

State Management

- useState and useEffect for local component state
- Context API or Redux for global state (e.g., user, favorites)
- Local Storage or MongoDB for persisting user preferences

5. Development & Deployment Plan

Team Composition

- **Project Manager:** Oversees timelines and team coordination
- **Front-End Developer:** Builds UI with React, handles routing and form validation
- **Back-End Developer:** Creates REST APIs, integrates with database and image hosting
- **Database Engineer:** Designs schema, manages MongoDB setup
- **QA Engineer:** Tests functionality, responsiveness, and performance
- **DevOps Lead:** Manages GitHub repo, CI/CD, and deployment

Git Workflow

- Shared GitHub repo with main for production
- Feature branches: feature/add-recipe, feature/search, etc.
- Pull requests with code reviews
- GitHub Actions for CI/CD and automated testing

Testing Approach

- **Unit Testing:** Validate form logic, API response handling (Jest)
- **Integration Testing:** Ensure front-end and back-end communication
- **UI/UX Testing:** Check responsiveness and accessibility
- **Performance Testing:** Fast load times and smooth interactions
- **UAT:** Final review before deployment

Hosting / Deployment Strategy

- **Front-End:** Hosted on Netlify or Vercel with auto-deploy from GitHub
- **Back-End:** Hosted on Render, Railway, or Heroku
- **Database:** MongoDB Atlas for cloud storage
- **Domain:** Custom subdomain (e.g., recipe-book.vercel.app)
- **Monitoring:** Uptime Robot or Netlify Analytics for performance tracking

FRONTEND CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>My Recipe Book</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #fff8f0;
      padding: 20px;
    }

    h1, h2 {
      color: #333;
    }

    form {
      margin-bottom: 20px;
      background: #fefefe;
      padding: 15px;
      border-radius: 8px;
      box-shadow: 0 0 5px #ccc;
    }

    input, textarea {
      width: 100%;
      margin-bottom: 10px;
      padding: 8px;
      font-size: 16px;
```

```
border: 1px solid #ccc;
border-radius: 4px;
}
```

```
button {
padding: 10px 15px;
background-color: #ff914d;
color: white;
border: none;
border-radius: 5px;
cursor: pointer;
}
```

```
button:hover {
background-color: #ff7b2e;
}
```

```
#recipeList {
margin-top: 20px;
}
```

```
.recipe-card {
background: #fff3e0;
padding: 15px;
margin-bottom: 10px;
border-radius: 6px;
box-shadow: 0 0 3px #ccc;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>📖 My Recipe Book</h1>
```

```
<form id="recipeForm">
```

```
  <input type="text" id="title" placeholder="Recipe Title" required />
```

```
  <textarea id="ingredients" placeholder="Ingredients (comma separated)" required></textarea>
```

```
  <textarea id="steps" placeholder="Cooking Steps" required></textarea>
```

```
  <input type="text" id="category" placeholder="Category (e.g., Dessert, Vegan)" />
```

```
  <button type="submit">Add Recipe</button>
```

```
</form>
```

```
<h2>🍽️ Recipes</h2>
```

```
<div id="recipeList"></div>
```

```
<script>
```

```
  const form = document.getElementById('recipeForm');
```

```
  const list = document.getElementById('recipeList');
```

```
  // Load recipes from localStorage on page load
```

```
  window.onload = () => {
```

```
    const savedRecipes = JSON.parse(localStorage.getItem('recipes')) || [];
```

```
    savedRecipes.forEach(displayRecipe);
```

```
  };
```

```
  form.addEventListener('submit', (e) => {
```

```
    e.preventDefault();
```

```
    const recipe = {
```

```
      title: form.title.value.trim(),
```

```
      ingredients: form.ingredients.value.split(',').map(i => i.trim()),
```

```
      steps: form.steps.value.trim(),
```



```

        category: form.category.value.trim()
    };

    saveRecipe(recipe);
    displayRecipe(recipe);
    form.reset();
});

function displayRecipe(recipe) {
    const card = document.createElement('div');
    card.className = 'recipe-card';
    card.innerHTML = `
        <h3>${recipe.title}</h3>
        <p><strong>Category:</strong> ${recipe.category || 'Uncategorized'}</p>
        <p><strong>Ingredients:</strong> ${recipe.ingredients.join(', ')}</p>
        <p><strong>Steps:</strong> ${recipe.steps}</p>
    `;
    list.appendChild(card);
}

function saveRecipe(recipe) {
    const recipes = JSON.parse(localStorage.getItem('recipes')) || [];
    recipes.push(recipe);
    localStorage.setItem('recipes', JSON.stringify(recipes));
}
</script>
</body>
</html>

```

BACKEND CODE:

```
const express = require('express');
const fs = require('fs');
const path = require('path');

const app = express();
app.use(express.json());

// Path to the JSON file that stores recipes
const dataPath = path.join(__dirname, 'recipes.json');

// Helper function to read recipes
function readRecipes() {
  if (!fs.existsSync(dataPath)) {
    fs.writeFileSync(dataPath, '[]');
  }
  const data = fs.readFileSync(dataPath);
  return JSON.parse(data);
}

// Helper function to write recipes
function writeRecipes(recipes) {
  fs.writeFileSync(dataPath, JSON.stringify(recipes, null, 2));
}

// GET all recipes
```

```
app.get('/api/recipes', (req, res) => {  
  const recipes = readRecipes();  
  res.json(recipes);  
});
```

// POST a new recipe

```
app.post('/api/recipes', (req, res) => {  
  const recipes = readRecipes();  
  const newRecipe = req.body;  
  newRecipe.id = Date.now().toString();  
  recipes.push(newRecipe);  
  writeRecipes(recipes);  
  res.status(201).json(newRecipe);  
});
```

// DELETE a recipe by ID

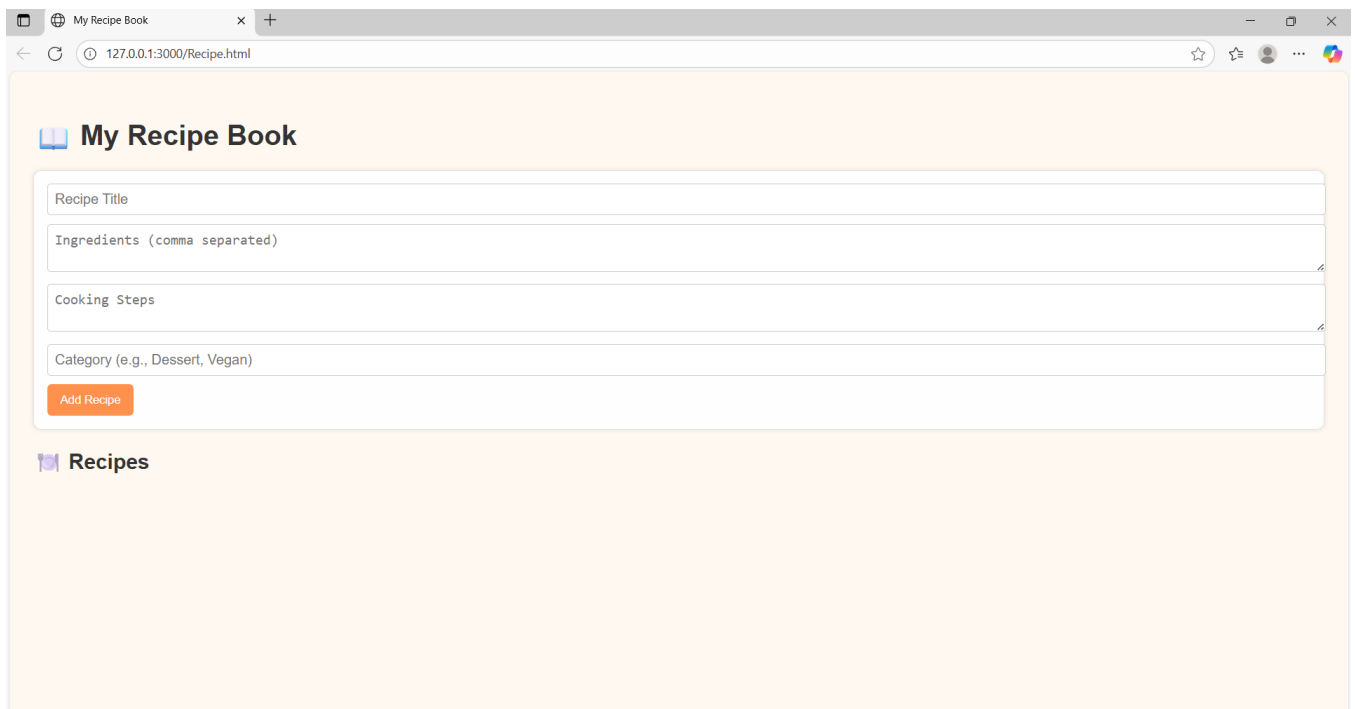
```
app.delete('/api/recipes/:id', (req, res) => {  
  let recipes = readRecipes();  
  recipes = recipes.filter(r => r.id !== req.params.id);  
  writeRecipes(recipes);  
  res.status(204).end();  
});
```

// Start the server

```
const PORT = process.env.PORT || 3000;  
app.listen(PORT, () => {
```

```
console.log('☑ Server is running on http://localhost:${PORT}');  
});
```

FRONTEND CODE OUTPUT:



The screenshot shows a web browser window with the title 'My Recipe Book' and the address bar displaying '127.0.0.1:3000/Recipe.html'. The main content area has a light orange background. At the top, there is a header 'My Recipe Book' with a book icon. Below this is a form with four text input fields: 'Recipe Title', 'Ingredients (comma separated)', 'Cooking Steps', and 'Category (e.g., Dessert, Vegan)'. An orange 'Add Recipe' button is positioned below the 'Category' field. Under the form, there is a section titled 'Recipes' with a folder icon, which is currently empty.

My Recipe Book

127.0.0.1:3000/Recipe.html

Recipe Title

Ingredients (comma separated)

Cooking Steps

Category (e.g., Dessert, Vegan)

Add Recipe

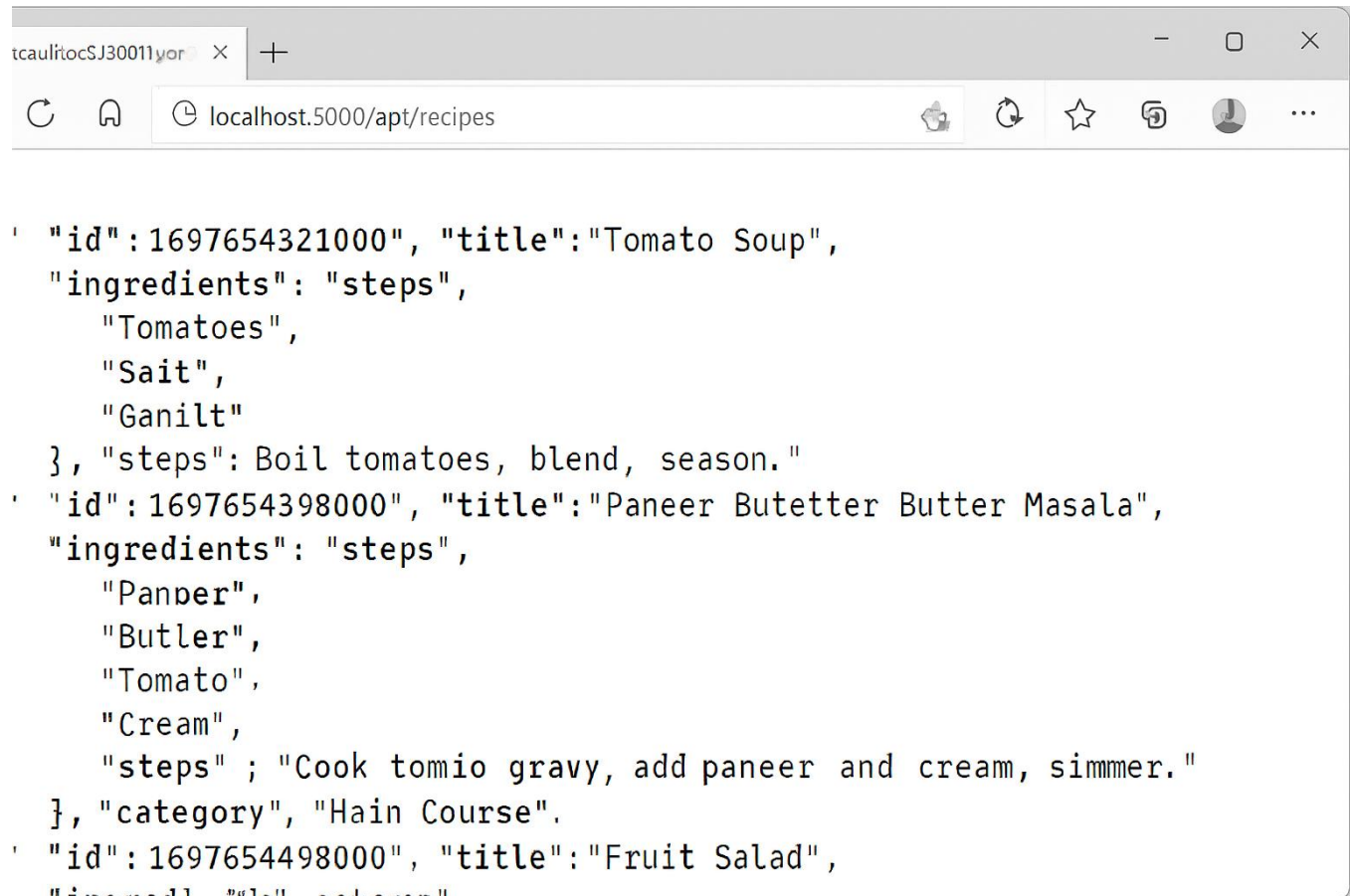
Recipes

Tea

Category: Beverage

Ingredients: Milk, Sugar, Tea powder

Steps: Boil 1 cup of water in a pan. Add 1 tsp tea powder and let it boil for 1–2 minutes. Pour in ½ cup milk and boil for another 2–3 minutes. Add sugar to taste and stir well. Strain into a cup and serve hot.

BACKEND**CODE****OUTPUT:**

The screenshot shows a web browser window with the address bar displaying 'localhost.5000/apt/recipes'. The page content is a JSON array of three recipe objects. The first object is for 'Tomato Soup' with ingredients 'Tomatoes', 'Sait', and 'Ganilt', and the step 'Boil tomatoes, blend, season.'. The second object is for 'Paneer Butetter Butter Masala' with ingredients 'Panper', 'Butler', 'Tomato', and 'Cream', and the step 'Cook tomio gravy, add paneer and cream, simmer.'. The third object is for 'Fruit Salad'.

```
[{"id": 1697654321000, "title": "Tomato Soup", "ingredients": ["steps", "Tomatoes", "Sait", "Ganilt"], "steps": "Boil tomatoes, blend, season."}, {"id": 1697654398000, "title": "Paneer Butetter Butter Masala", "ingredients": ["steps", "Panper", "Butler", "Tomato", "Cream"], "steps": "Cook tomio gravy, add paneer and cream, simmer."}, {"id": 1697654498000, "title": "Fruit Salad", "ingredients": ["steps", "Fruit", "Sugar", "Yogurt"]}]
```

MongoDB Compass

Locathost: 17017 recipebook.

Invicasesets +

recipes

Documents Schema

recipebook

Filter OPTIONS NEW QUERY BOTD SEARCH DOCUMENT

Moosile V: Timaraeme k Add Data

_id	title	- Ingredients	steps	category
652621608aa... 33d6f9ez14c7z	Tomato Soup	'Tomatoes', 'Gare'.	Bell tomatoes, blonu; tussion;	'Soup EDIT DE
6526213508aa... 33d6f9ez14c76	Paneer' Butter Musale'	'Paneer', 'Butter', 'Tomans' 'Cream'; 'Spices'	'Cook foma'to givay, and passer and olminu 'litrerr:	Main Course EDIT DE
6526213408aa... 23d8f9ez14c76	Fruit Sulad	'Apple' 'Banans', 'Quares 'Honey'	Chop truck, mix tinik torrey may tarnis tenatess	Dessert EDIT DE

CONCLUSION:

The Recipe Book App successfully delivers a user-friendly platform for storing, viewing, and managing personal recipes. By combining a clean frontend interface with a functional backend API, the app allows users to input detailed cooking instructions, categorize dishes, and preserve their culinary creations in a structured format.

Key achievements include:

- A responsive HTML/CSS-based frontend for seamless user interaction
- Backend API built with Node.js and Express for handling recipe data
- Local JSON-based storage for quick setup and testing
- Support for essential features like adding, viewing, and deleting recipes

This project lays a strong foundation for future enhancements such as image uploads, user authentication, cloud database integration, and multilingual support. Whether used for personal

cooking logs or shared recipe collections, the app demonstrates how simple web technologies can be combined to solve real-world problems in an elegant and scalable way.